

RESUMEN LENGUAJES DE PROGRAMACIÓN

Lenguajes de Programación: Conceptos y Paradigmas	1
Definición Técnica Literal.....	1
Concepto de Entorno de Desarrollo (IDE)	2
Clasificación según Ejecución	2
Paradigmas de Programación	3
Representación de Tipos de Datos	4
Tipos Simples (Datos Básicos).....	4
Datos Derivados y Estructurados	5
Operadores y Expresiones.....	5
Tipos de Operadores	5
Precedencia de Operadores.....	5
Instrucciones Condicionales e Iterativas (Control de Flujo).....	5
Estructuras Alternativas (Decisión).....	5
Estructuras Repetitivas (Bucles/Iteraciones)	6
El Bucle Infinito.....	6
Recursividad	6
Procedimientos, Funciones y Parámetros.....	7
Definiciones.....	7
Paso de Parámetros.....	7
Vectores y Registros	8
Estructura de un Programa	8
Elementos Adicionales	8

Lenguajes de Programación: Conceptos y Paradigmas

Definición Técnica Literal

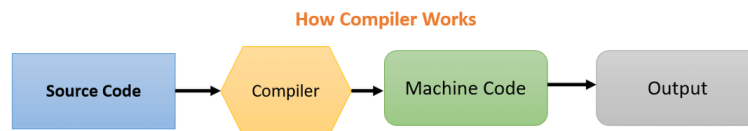
- **Programa:** "Serie o secuencia de instrucciones que el ordenador debe ejecutar para realizar la tarea prevista por el programador".

- **Lenguajes de Programación:** "Conjunto de instrucciones... que deben traducirse de un lenguaje comprensible por los humanos a un lenguaje comprensible para la máquina".

Concepto de Entorno de Desarrollo (IDE)

Es una aplicación que proporciona un conjunto de herramientas integradas para facilitar el desarrollo de software. Sus componentes principales son:

- **Editor de texto:** Donde se escribe el código fuente (suele incluir resaltado de sintaxis).
- **Compilador/Intérprete:** El motor que traduce o ejecuta el código (visto en el punto anterior).



- **Depurador (Debugger):** Herramienta esencial para ejecutar el programa paso a paso, inspeccionar variables y detectar errores de lógica.
- **Gestor de proyectos/Build:** Para organizar archivos y automatizar la creación del ejecutable.

Clasificación según Ejecución

- **Lenguajes Compilados:** El código de alto nivel se traduce íntegramente a código máquina (binario nativo) antes de ejecutarse. Son los más rápidos.
 - **Preprocesador:** Gestiona directivas iniciales (ej. #include en C).
 - **Ensamblador:** Traduce el código fuente a código objeto.
 - **Linker (Enlazador):** Une el código objeto con librerías para generar el archivo ejecutable.

- **Lenguajes Interpretados:** El código se lee y ejecuta línea a línea por un intérprete en tiempo de ejecución. Suelen permitir tipado dinámico pero son más lentos.
 - **Tipado Estático vs. Dinámico:** Si el tipo de variable se comprueba en compilación o en ejecución.
 - **Tipado Fuerte vs. Débil:** Si el lenguaje permite (o no) realizar operaciones entre tipos distintos sin conversión explícita.
 - **Recolector de Basura (Garbage Collector):** Proceso automático que libera la memoria de objetos que ya no se usan (típico de Java, frente a la gestión manual de C con punteros).
- **Lenguajes de Compilación Intermedia (Híbridos):** No generan código máquina directo, sino un código intermedio.
 - **Bytecode:** Código binario (ej. Java) que necesita una Máquina Virtual para ejecutarse en cualquier sistema.
 - **CIL:** El equivalente al Bytecode en el ecosistema .NET.
- **Transpilador:** Traduce código de un lenguaje de alto nivel a otro lenguaje de alto nivel (ej. de TypeScript a JavaScript). No es un híbrido entre intérprete y compilador.

Paradigmas de Programación

- **Programación Imperativa:** "Define estados y cambios sobre esos estados. Lo contrario de la programación declarativa. Ejemplos: C, Java, PHP". Basado en el modelo Von Neumann, donde un conjunto de operaciones primitivas realizan una ejecución secuencial.
- **Programación Declarativa:** "Definen condiciones, excepciones, ecuaciones que describen un problema y la solución. No existe un orden de evaluación prefijado. Ejemplo: SQL".
 - **Funcionales:** "Definen los programas como funciones matemáticas... Una función puede ser argumento de otra. Ejemplos: Lisp, Scheme".
 - **Lógicos:** "Se basan en el cálculo de predicados... permite que un ordenador derive en soluciones inteligentes. Ejemplo: Prolog".
- **Programación Orientada a Objetos (POO):** "Derivada de la Imperativa. Ejemplos: Smalltalk, C++, Java".

- **Encapsulamiento:** Ocultar los detalles internos de un objeto y proteger su estado.
- **Herencia:** Un objeto (subtipo) adquiere propiedades de otro (supertipo).
- **Polimorfismo:** Capacidad de una entidad de referenciar a instancias de distintas clases y responder de forma diferente según el caso.
- **Abstracción:** Capacidad de ignorar los detalles irrelevantes y centrarse en lo esencial.

Enfoque CPS: El tribunal suele preguntar por la diferencia entre lenguajes compilados e interpretados, así como identificar qué lenguajes pertenecen a cada paradigma (ej: SQL como declarativo). Un distractor frecuente es asignar a C++ como lenguaje de "segunda generación" cuando es de tercera/cuarta.

Clasificación histórica completa:

- **1GL (Primera):** Lenguaje máquina (binario puro).
- **2GL (Segunda):** Lenguaje ensamblador (uso de mnemotécnicos).
- **3GL (Tercera):** Lenguajes de alto nivel orientados a procedimientos (C, Pascal, Fortran).
- **4GL (Cuarta):** Orientados a la gestión y bases de datos (SQL, lenguajes de informes).
- **5GL (Quinta):** Inteligencia artificial y procesamiento de lenguaje natural (Prolog, Lisp).

Representación de Tipos de Datos

Tipos Simples (Datos Básicos)

- **Numéricos:** "Entero, Real".
 - **Dato de Doble Precisión (IEEE 754-2008):** "Una aproximación de 64 bits de un número real".
- **Carácter:** "Corresponde a un valor numérico entero sin signo, siguiendo un determinado código (ASCII, EBCDIC, etc.)".
- **Lógico (Booleano):** "Se emplean para representar únicamente dos valores: 1 ó 0, cierto o falso".

Datos Derivados y Estructurados

- **Puntero:** "Dato derivado... se emplea para contener la dirección de memoria de otra variable".
- **Constante:** "Es una entidad cuyo valor no puede ser modificado por el programa".
- **Variable:** Contenedor que puede variar su valor.
 - **Alcance de una variable:** "Ámbito de visibilidad de la misma dentro del programa".

Operadores y Expresiones

Tipos de Operadores

Categoría	Símbolos Literales
Aritméticos	+ (Suma), - (Resta), * (Multip.), / (División), \ (Div. entera), % o MOD (Módulo)
Relacionales	=, == (Igual), !=, <> (Distinto), <, <=, >, >=
Lógicos	! o NOT (Negación), && o AND (Conjunción), `

Precedencia de Operadores

- "Cuando la multiplicación y la división o la suma y la resta aparecen juntos, son evaluados de izquierda a derecha".
- **Concatenación:** "Se ejecuta después de los operadores aritméticos y antes que los de comparación".

Enfoque CPS: Es común encontrar preguntas de lógica booleana con valores asignados. Ejemplo: Not (a==c) and (c>b) con a=1, b=2, c=3. El resultado correcto es True.

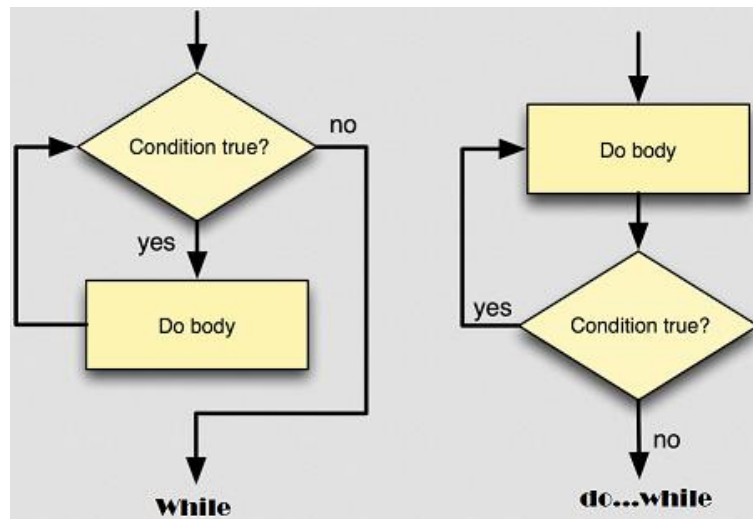
Instrucciones Condicionales e Iterativas (Control de Flujo)

Estructuras Alternativas (Decisión)

- **Si (if) simple:** Ejecuta una acción si se cumple la condición.
- **Si (if) doble:** Incluye una cláusula Sino (else) para cuando no se cumple.
- **Según sea / Switch (CASE):** "Sirve para devolver un valor y no para ejecutar una instrucción" (según PL/SQL).

Estructuras Repetitivas (Bucles/Iteraciones)

- **Mientras (While):** "Conjunto de instrucciones que se ejecutarán repetidamente mientras se cumplan unas determinadas condiciones". Puede no ejecutarse nunca si la condición es falsa inicialmente.
- **Repetir (Do...While / Repeat...Until):** "Estructura de control que al menos siempre se ejecuta una vez".



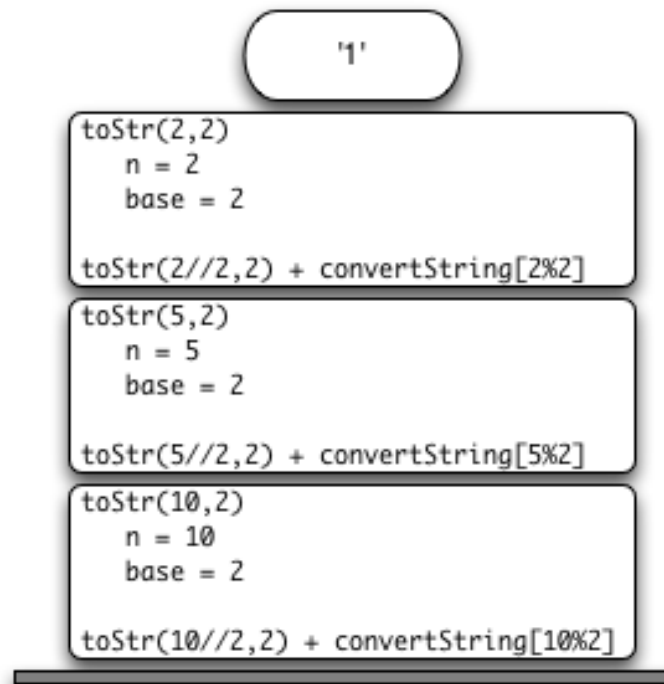
- **Para (For):** Bucle con contador de un valor mínimo a un máximo con incremento específico.

El Bucle Infinito

- **CPS evalúa fragmentos de código para identificar bucles sin fin.**
- **Código:** `integer x=0; while x<100 do (x=x+1; print x; x=x-1;)`. **Resultado:** "Este programa es un bucle infinito".

Recursividad

- **Definición Literal:** "Consiste en que un programa puede llamarse a sí mismo".
- **Contexto:** Se utiliza en algoritmos tipo "divide y vencerás".



Procedimientos, Funciones y Parámetros

Definiciones

- **Procedimiento:** "Subprograma que realiza una tarea específica... Tanto la entrada como la devolución de resultados... se realizará a través de los parámetros".
- **Función:** Similar al procedimiento pero devuelve un valor directo como resultado de su ejecución.
- **Instrucciones Complejas:** "Bloque de acciones agrupadas en subrutinas, subprogramas, funciones o módulos".

Paso de Parámetros

- **Por Valor:** "Envía una copia del valor... no se podrá alterar el contenido de la variable".
- **Por Referencia:** "Se pueden emplear como parámetros de entrada/salida".



Vectores y Registros

- **Vector o Array:** "Tipo de dato estructurado que permite almacenar un conjunto de datos homogéneo donde cada elemento se almacena de forma consecutiva en memoria".
- **Registro (Record/Struct):** "Estructura estática de datos... tipo de datos que se compone de datos más simple" (ej: nombre, apellidos, dirección).

Estructura de un Programa

Un programa se divide en tres bloques diferenciados:

1. **Entrada de datos:** Instrucciones que toman datos de un periférico externo y los depositan en la memoria principal.
2. **Proceso o algoritmo:** Instrucciones encargadas de procesar la información o datos pendientes de elaborar depositados en memoria.
3. **Salida de datos:** Instrucciones que toman los resultados de la memoria y los envían a un dispositivo externo.

Elementos Adicionales

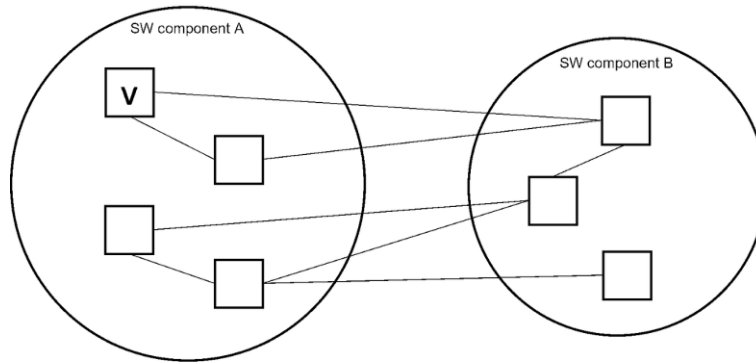
- **Comentarios:** "Líneas de texto que el compilador o el intérprete no consideran como parte del código". En C se introducen entre /* y */.
- **Directivas de Preprocesador:** Interpreta las directivas. En C se distinguen por el carácter #.

Enfoque CPS General: CPS suele preguntar por la Cohesión ("Relación funcional entre los distintos elementos que componen un

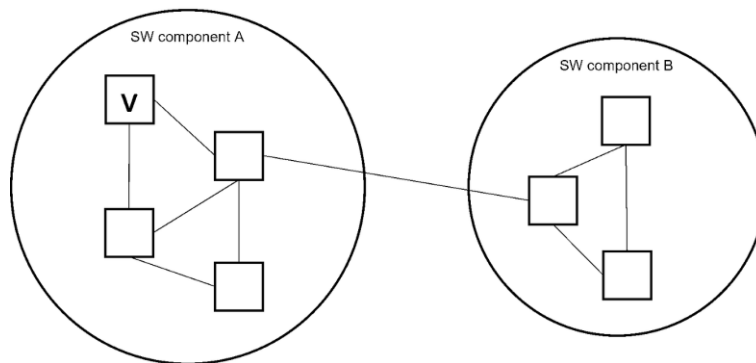
módulo. Debe ser alto") y el Acoplamiento ("Grado de interdependencia entre los distintos módulos. Debe ser bajo").

- **Cohesión (Alta = Buena): "Cada módulo sabe hacer muy bien lo suyo y solo lo suyo".**

Bad: High coupling, low cohesion



Good: Low coupling, high cohesion



- **Acoplamiento (Bajo = Bueno): "Los módulos son independientes; si rompes uno, no se rompen todos los demás".**

